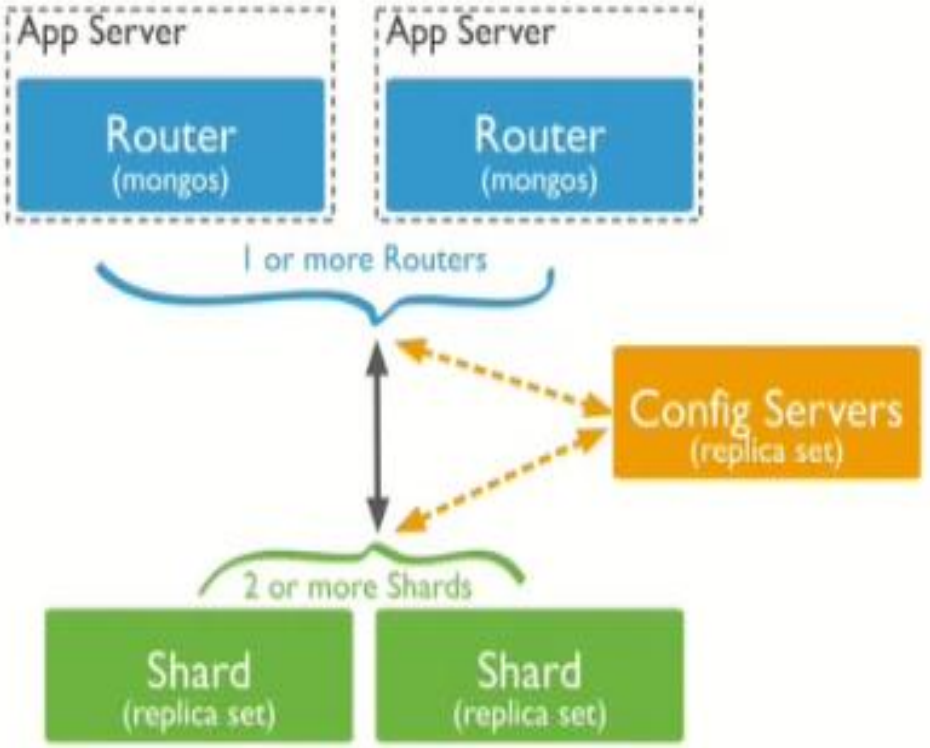
Mongodb: Sharding

Avantages du sharding MongoDB

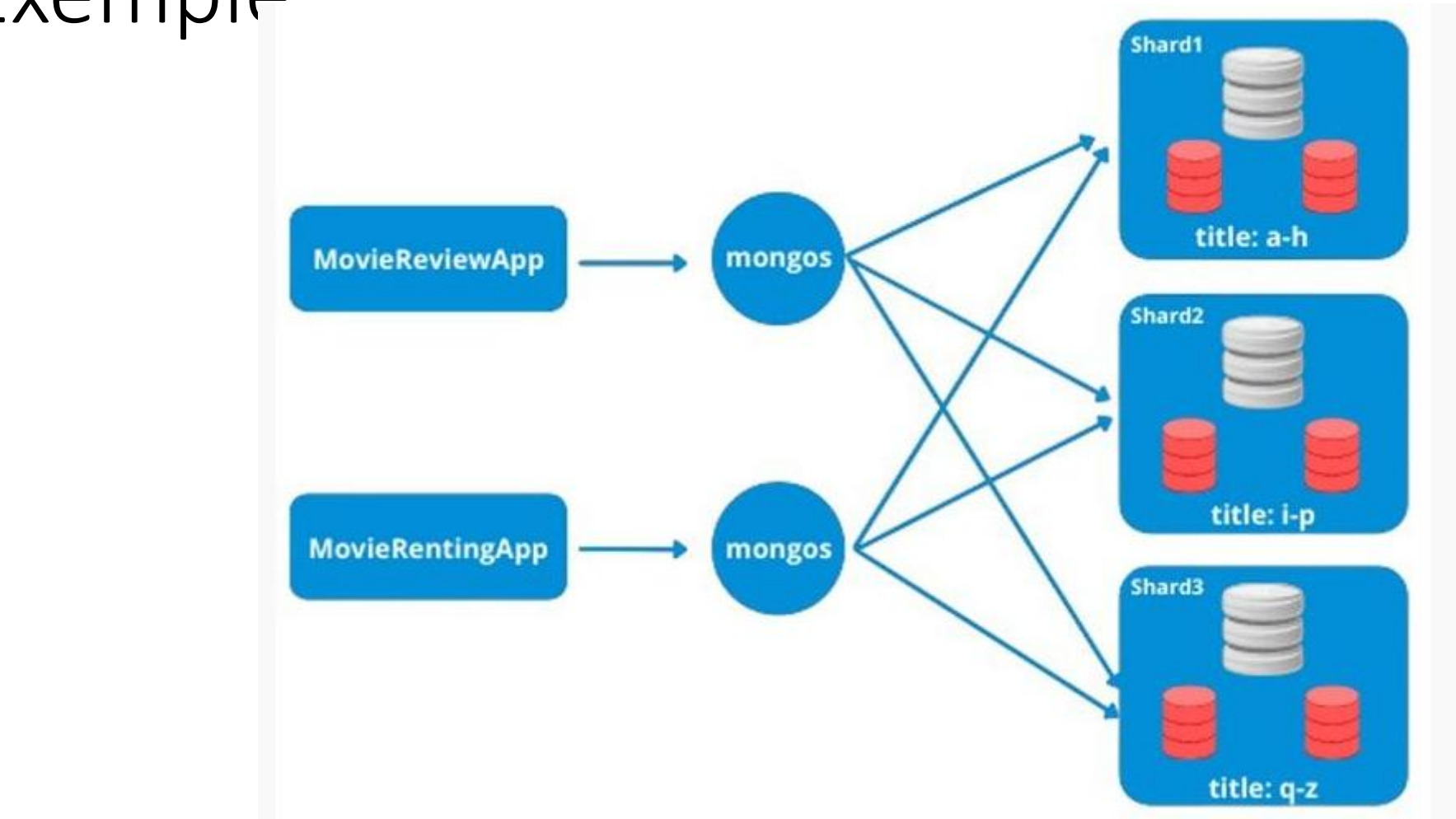
- Capacité de stockage
- Lecture/écriture
- Haute disponibilité
- Protection contre les pannes
- Géodistribution et performances

Architecture de partitionnement



La robustesse du *cluster* est gérée par 3 types de nœuds :
1-Des routeurs : **mongos**
2-Des serveurs de configuration : **Config Server**
3-Des serveurs de données : **Shard**

Exemple


MovieReviewApp
MovieRentingApp
mongos
mongos
Shard1
title: a-h
Shard2
title: i-p
Shard3
title: q-z

Exemple

chaque application va se connecter à un mongos et envoyer sa requête.

- Si la requête contient le champ « title »,
-Si en revanche la requête ne contient pas le champ « title »,

Pour améliorer la disponibilité et la scalabilité de votre application **……**

Les ConfigServers

Pour que mongos soit capable de savoir dans quel shard se trouve la donnée recherchée…
La répartition des données étant faite dynamiquement……

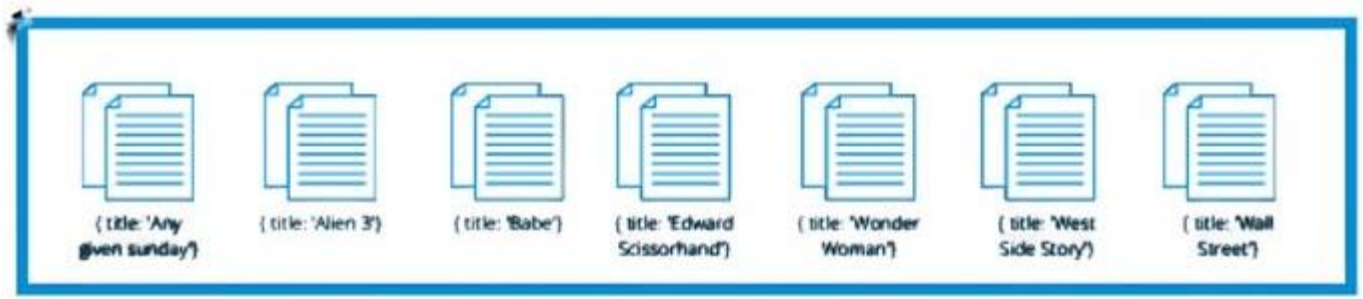
❑ Clé de partition

- Une Clé de partition est utilisée pour distribuer les collections de documents en des partitions
- Une clé de partition est composée d'un champ qui existe dans les documents de la collection
- On fait le choix de la clé lors du partitionnement
- Une collection partitionnée possède une seule clé

❑ Chunks

- Mongo DB divise les données partitionnées en des chunks
- Mongo DB divise chunk chaque fois qu'il augmente de taille lorsqu'il dépasse la taille fixée lors de la configuration qui es par défaut 64 megabyets
- Mongo DB migre les chunks entre partitions lorsqu'une partition possède assez de chunks

Dès l'instant où on définit une clé de sharding, mongodb crée un premier Chunk contenant l'ensemble des documents:



Comment configurer le sharding MongoDB étape par étape ?

Pour configurer le sharding MongoDB étape par étape, suivez les instructions suivantes :

Étape 1 : lancé deux serveurs de configuration (**--configsvr**) en ReplicaSet (**configReplSet**)

```
mongod --configsvr --replSet configReplSet --port 27019 --dbpath /data/conf1
mongod --configsvr --replSet configReplSet --port 27020 --dbpath /data/conf2
```

- Dans une console distincte, connectez-vous à l'un de ces serveurs et initiez le ReplicaSet :

```
mongo --port 27019
rs.initiate();
```

- Ajoutez le deuxième serveur de configuration au ReplicaSet :

```
rs.add("local:27020");
```

Étape 2 : Création des shards

Lancez les serveurs shard en ReplicaSet :

```
mongod --shardsvr --replSet sh1 --port 27031 --dbpath /data/sh1
mongod --shardsvr --replSet sh2 --port 27032 --dbpath /data/sh2
```

Initiez le ReplicaSet de chaque shard dans une console distincte :

```
mongo --port 27031 --eval "rs.initiate()"
mongo --port 27032 --eval "rs.initiate()"
```

Étape 3 : Lancement du mongos (serveur de distribution) et connexion des shards
Lancez le serveur mongos en spécifiant le serveur de configuration :

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

Ajoutez les shards un par un au niveau du mongos en mode console :

```
mongo --port 27017
sh.addShard("sh1/localhost:27031");
sh.addShard("sh2/localhost:27032");
```

Étape 4 : Distribution de la base de données

Définissez la collection que vous souhaitez distribuer. Pour cela, créez une base de données "testDB" et une collection "test" :

```
use testDB;
sh.enableSharding("testDB");
db.createCollection("test");
db.test.createIndex({"_id": 1});
sh.shardCollection("testDB.test", {"_id": 1});
```

Pour vérifier l'état du sharding, exécutez la commande suivante dans une console :

```
mongo --port 27017 --eval "sh.status()"
```